```
import itertools

# Define the target goal configuration (0 represents the empty space)
GOAL_STATE = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def get_moves(state):
    """Generates all valid successor states and their transition directions."""
    moves = []
    zero_idx = state.index(0)
    row, col = divmod(zero_idx, 3)

    # Direction actions mapped to index changes in a 1D tuple
    directions = {
        'UP': (-1, 0, -3),
        'DOWN': (1, 0, 3),
        'LEFT': (0, -1, -1),
        'RIGHT': (0, 1, 1)
    }

    for direction, (dr, dc, move_idx) in directions.items():
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            # Create a mutable list variant to swap elements safely
            neighbor = list(state)
            neighbor[zero_idx], neighbor[zero_idx + move_idx] = neighbor[zero_idx + move_idx], neighbor[zero_idx]
            moves.append((tuple(neighbor), direction))

    return moves

def depth_limited_search(state, depth, current_path):
    """Performs a depth-limited search looking for the goal state."""
    if state == GOAL_STATE:
        return []
    if depth <= 0:
        return None

    current_path.add(state)

    for next_state, direction in get_moves(state):
        if next_state not in current_path:
            result = depth_limited_search(next_state, depth - 1, current_path)
            if result is not None:
                return [direction] + result

    current_path.remove(state)
    return None

def iterative_deepening_search(initial_state):
    """Increments the depth limit sequentially until a path is discovered."""
    for depth in itertools.count():
        current_path = set()
        result = depth_limited_search(initial_state, depth, current_path)
        if result is not None:
            return result

def print_board(state):
    """Helper utility to display the state in a scannable 3x3 layout."""
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]} ")
    print("-" * 11)

# --- Execution ---
if __name__ == "__main__":
    # Example starting layout (Solvable within 5 moves)
    initial_puzzle = (1, 2, 3, 4, 0, 6, 7, 5, 8)

    print("Initial Layout:")
    print_board(initial_puzzle)

    print("Searching for solution...")
    solution_path = iterative_deepening_search(initial_puzzle)

    print(f"\nSolution discovered in {len(solution_path)} steps!")
    print("Sequence:", " -> ".join(solution_path))
```

```
Initial Layout:
 1 2 3
 4 0 6
 7 5 8
-----------
Searching for solution...

Solution discovered in 2 steps!
Sequence: DOWN -> RIGHT
```